

17-214/514: Agentic Software Development

Modularity and Anti-Patterns

Hammad Ahmad and Claire Le Goues • Monday, September 21, 2026 • **Lecture**

Administrivia

- Midterm 1 is on Wednesday. In this room, at 11am.
 - 80-minute exam, pen and paper.
 - Scope: everything we have seen so far, not including today's material.
 - You can bring 4 pages of US letter-size cheat sheet, front and back.
 - Sample questions were posted over the weekend.
- A1 grades were released. Regrade requests are open until Friday (11:59pm).
- There was a question on Piazza about how much you are expected to read design decisions. More on that later today.

What did we talk about last time?

- Change happens. Design so the likely changes stay cheap and local.
- Low coupling, high cohesion. Talk through interfaces, not internals.
- GRASP heuristics (Information Expert, Creator, Controller) decide where behavior lives.
- For agents, coupling is reach and cohesion is one focused context.
- Decouple what varies. Over-decoupling has its own cost.

Learning goals for today

- Identify common design anti-patterns and explain why they hurt maintainability
- Critique a design using shared software engineering vocabulary
- Recognize the characteristic ways **generated** code goes wrong
- Use anti-pattern names as a checklist when reviewing an agent's output
- Distinguish a real design problem from a superficial smell match

The tests pass. Why the bad feeling?

The agent's feature works. The suite is green. Then you decide to open the file.

- No test fails. Nothing you could point to is wrong.
- You suspect the next change here will be harder than it should be.
- **Today:** Names for what is possibly bothering you, and how to use them in a review.

A vocabulary for design problems

Why name the smells?

- "This code feels off" is hard to act on.
- "This is a God class" tells a reviewer what to look at.
- A shared name lets two engineers, or an engineer and an agent, agree on what is wrong.
- An agent hands you a few hundred lines. You can ask "Does it seem off to you?", or you can ask "Any God class? Feature envy? Hidden dependencies?"
- (A smell can make a future change harder whether or not anyone reads the code. The agent making that change uses more tokens, and its edit can reach further.)

Where the names come from

- Kent Beck coined "code smell". Martin Fowler's *Refactoring* (1999) carries the catalog.
- "Anti-pattern" is the sibling term for a common design that looks reasonable and causes trouble. The smell is how we notice it.
 - Most people use the two names loosely.
- Some catalogs group the smells into families (bloaters, couplers, dispensables).
- Agents learned the same names. That is why "there is a God class here" works as a prompt.

Code Smells

– What? How can code "smell"??
– Well it doesn't have a nose... but it definitely can stink!



Bloaters

Bloaters are code, methods and classes that have increased to such gargantuan proportions that they are hard to work with. Usually these smells do not crop up right away, rather they accumulate over time as the program evolves (and especially when nobody makes an effort to eradicate them).

§ Long Method

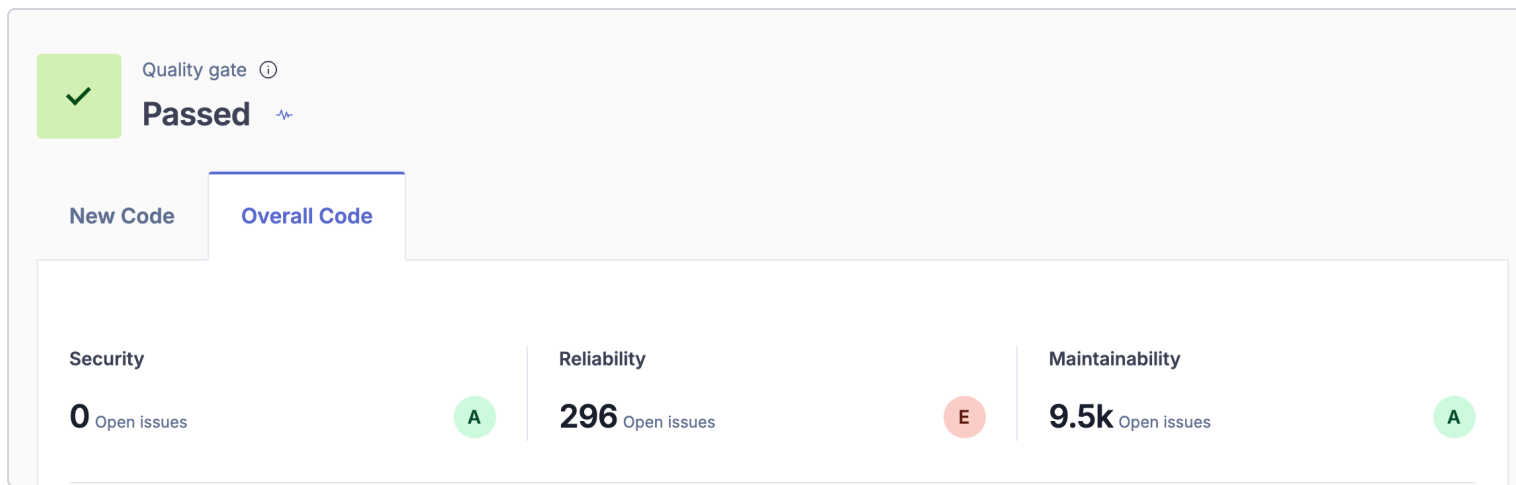
§ Primitive

§ Data Clumps

refactoring.guru/refactoring/smells

A smell is not a bug

- A code smell is a **symptom**. The code can pass every test and still smell.
- It suggests the structure will fight us on the next change (or at a future point).
- Smells are mostly about modifiability and testability, not correctness.



sonarcloud.io/summary/overall?id=apache_groovy

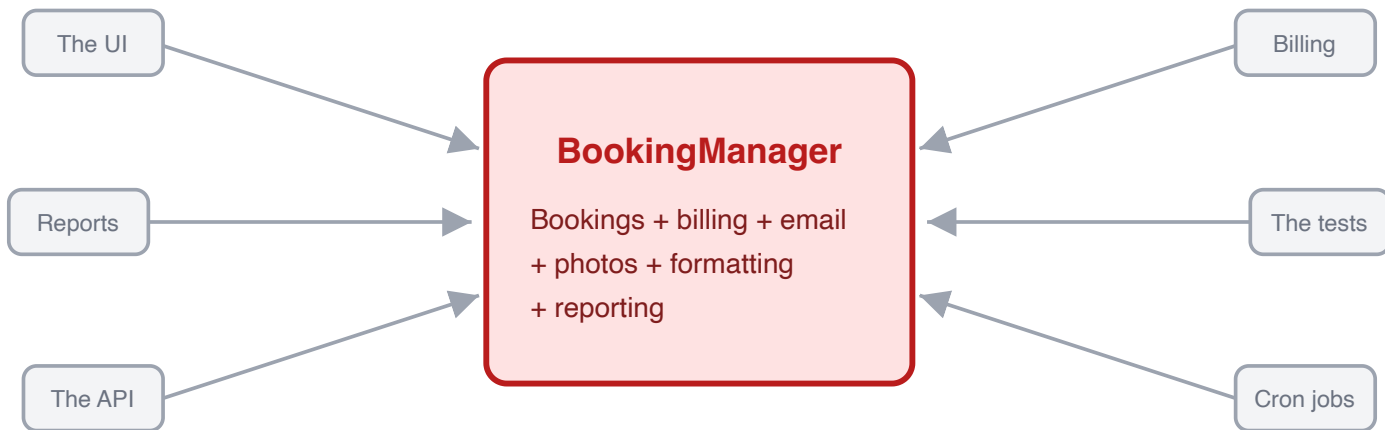
Where smells come from

- Designed badly. The structure was wrong on day one, and the code inherited it.
- Evolved badly. A reasonable design, plus two years of "just one more field here".
- Generated fast. Reasonable-looking code, produced without a view of the whole, at machine speed.

The classic smells

God class

- One class that knows too much and does too much.
- It accreted responsibilities until nobody can say what it is **for**.
- Every feature touches it, so most changes risk breaking something unrelated.
 - Often the first place an agent adds to when you ask for a feature.



Consider BookingManager

```
class BookingManager {  
    Booking book(...) { ... }  
    void chargeCard(...) { ... } // billing  
    void sendConfirmation(...) { ... } // email  
    void resizeRoomPhoto(...) { ... } // images  
    String formatDate(...) { ... } // formatting  
    void exportCsv(...) { ... } // reporting  
}
```

This class has six reasons to change. Split it along its responsibilities.

Splitting BookingManager

```
class BookingService { Booking book(...) { ... } }  
class BillingService { void chargeCard(...) { ... } }  
class Notifier       { void sendConfirmation(...) { ... } }
```

- Each class now has one reason to change and one audience.
- Each can be tested, reused, and handed to an agent on its own.

java.util.Calendar was guilty of this

- One mutable class holds a point in time, a calendar system, a time zone, and date arithmetic.
- Its docs list about 50 `int` constants. Months, field numbers, and AM/PM values share one list.
- Java 8 split it into `java.time`, where each class has one job.
 - It is the same move we just made on `BookingManager`.
- Is it a God class? Arguably. It has low cohesion either way.

Field Summary

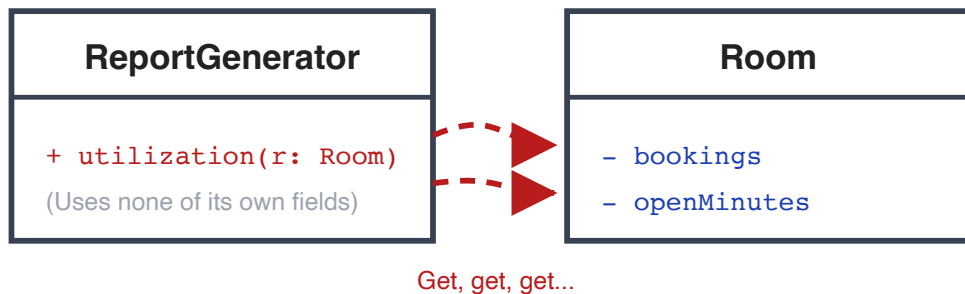
Fields

Modifier and Type	Field	Description
static final int	ALL_STYLES	A style specifier for <code>getDisplayNames</code> indicating names in all styles, such as "January" and "Jan".
static final int	AM	Value of the <code>AM_PM</code> field indicating the period of the day from midnight to just before noon.
static final int	AM_PM	Field number for get and set indicating whether the HOUR is before or after noon.
static final int	APRIL	Value of the <code>MONTH</code> field indicating the fourth month of the year in the Gregorian and Julian calendars.
protected boolean	areFieldsSet	True if <code>fields[]</code> are in sync with the currently set time.

docs.oracle.com, java.util.Calendar

Feature envy

- A method more interested in **another** object's data than its own.
- It keeps reaching across the boundary to do work that belongs over there.
- This means the behavior lives in the wrong class.



Move utilization() next door, where the data is
(Information Expert)

Consider ReportGenerator

```
class ReportGenerator {  
    double utilization(Room r) {  
        // reaching into Room, over and over,  
        // to compute Room's own statistic  
        long booked = r.getBookings().stream()  
            .mapToLong(b -> b.minutes()).sum();  
        return booked / (double) r.getOpenMinutes();  
    }  
}
```

This is `Room`'s job. Move `utilization()` onto `Room`, where the data lives (Information Expert).

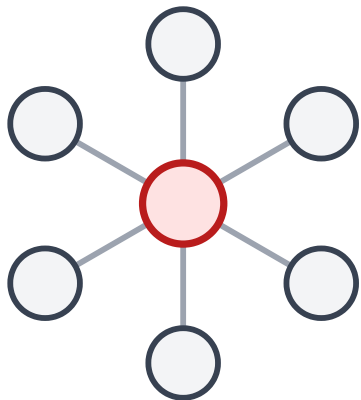
Inappropriate intimacy

```
class Room {  
    List<Booking> bookings;  
    void close() { for (Booking b : bookings) b.status = "CANCELLED"; }  
}  
class Booking {  
    String status; Room room;  
    void cancel() { room.bookings.remove(this); status = "CANCELLED"; }  
}
```

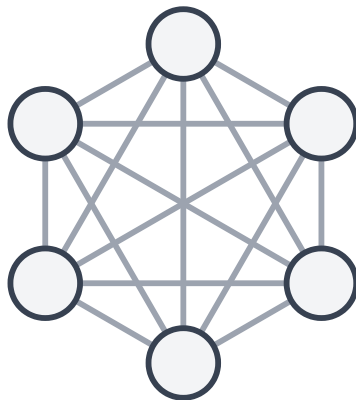
- Two classes that reach into each other's fields, in both directions.
- Feature envy reaches one way. This smell reaches both ways, so neither class can change alone.
- A fix is for each class to change its own state behind a method.
 - For example, `booking.cancel()` and `room.release(booking)`.

Excessive coupling

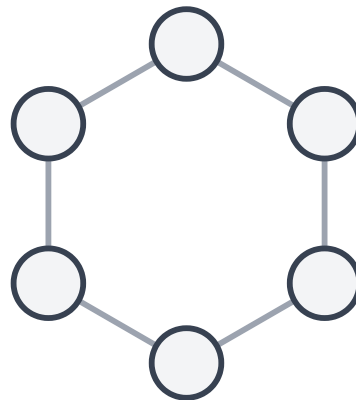
- Everything depends on everything. One change ripples across the codebase.
- **Shotgun surgery:** A small change forces edits in many files.



A. One hub
six edges, all via the hub



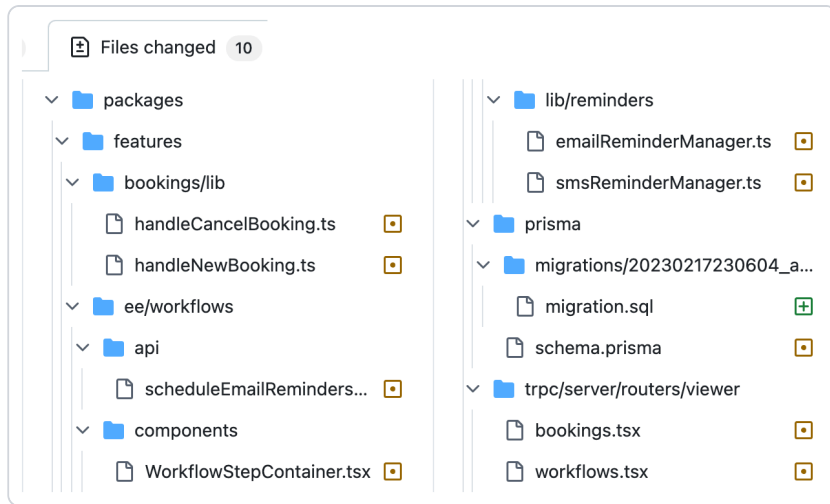
B. Everyone to everyone
fifteen edges



C. Neighbors only
six edges, nothing central

The ten-file diff from last time

- One sentence of request. Ten files across three packages.
- The same reminder-deleting call was copied into seven places, so the fix had to visit all seven.
- This is shotgun surgery.
 - Some smells show up only in a diff instead of any one file.



<https://github.com/calcom/cal.com/pull/7232/files>

Divergent change

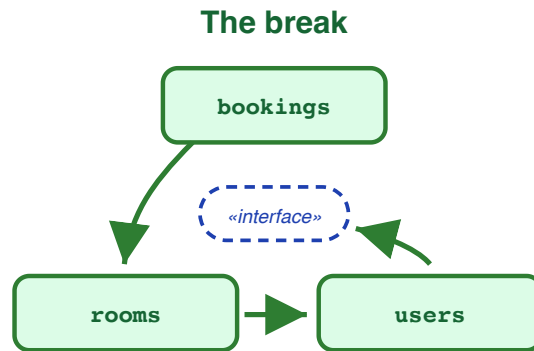
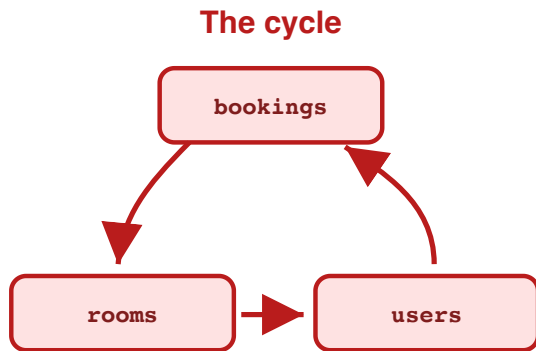
This is the opposite case, where one file changes for five unrelated reasons.

```
$ git log --oneline -- src/BookingManager.java
a41f2c0 add a CSV export column for finance
9b73e1d switch email provider
c02d8aa resize photos for the new mobile layout
5e91b77 fix an off-by-one in the overlap check
d7c3f04 change the date format for the EU rollout
```

- The five commits are about finance, email, photos, booking rules, and formatting.
- Shotgun surgery is one change in many files. Divergent change is many changes in one file.
- Both show up in the history (and not in any single read of the code).

Cyclic dependencies

- `bookings` imports `rooms`, `rooms` imports `users`, `users` imports `bookings`.
- A cycle has no safe starting point. We can't change, test, or understand one module on its own.
- Agents can add cycles without us noticing, since each new import looks reasonable on its own.
- To break one, extract the shared piece, or put an interface at one edge and point the arrow one way.



Hidden dependencies

- A function that needs something its signature never mentions.
- A global, a singleton, an environment variable, the system clock.
- We can't see it, so we can't reason about it or reuse it, and testing it means setting up everything it touches.
 - The testing half of this is controllability (from Week 2).

Consider book()

```
Booking book(User u, Room r, Slot s) {  
  if (s.start().isBefore(LocalDate.now())) reject(); // the clock  
  if (GlobalConfig.get("bookings_open")) { ... }      // global state  
  ...  
}
```

What the signature admits



book(User, Room, Slot)

"that's everything I need,
promise"

LocalDate.now()

What it also needs

GlobalConfig

Two inputs the signature never mentions

Long method

```
async function makeBooking(req: Request): Promise<Booking> {  
  if (!req.user || !req.room) throw new Error("missing fields");  
  if (req.start >= req.end) throw new Error("bad interval");  
  const room = await rooms.find(req.room);  
  for (const b of room.bookings)  
    if (b.start < req.end && req.start < b.end) throw new Error("taken");  
  let price = room.rate * hours(req.start, req.end);  
  if (isPeak(req.start)) price *= 1.5;  
  if (req.user.plan === "member") price *= 0.9;  
  const booking = await bookings.insert({ ...req, price });  
  await mailer.send(req.user.email, confirmation(booking));  
  log.info(`booked ${booking.id} for ${req.user.id}`);  
  return booking;  
}
```

- Validation, availability, pricing, persistence, notification, logging. Six jobs in one method.
- Read it as a table of contents. Each block could be a named function.

Primitive obsession

```
// two raw longs  
void book(User u, Room r, long startMillis, long endMillis) { ... }  
  
// a real type  
void book(User u, Room r, Slot slot) { ... }
```

- Raw primitives carry no rules. Nothing stops `end` from coming before `start`, for example.
- A `Slot` type can enforce its own invariants and read like the domain.
- `startMillis` and `endMillis` always travel together. Values like that are called a **data clump**.

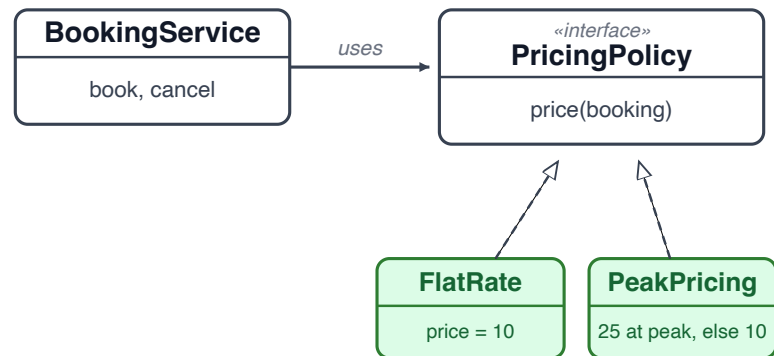
Type checks instead of polymorphism

```
double price(PricingPolicy p, Booking b) {  
    if (p instanceof FlatRate f)      return f.rate;  
    if (p instanceof PeakPricing pp) return pp.priceAt(b.slot());  
    return 0;  
}
```

- Every new policy means another branch here, and in every method like it.
- Ask the object instead: `p.price(b)`, with each policy answering for itself.
- The general move is **polymorphism**. Strategy is one pattern built on it.

What Strategy looks like

- The service depends on an interface, never on a particular rule.
- Each rule is its own box below.
- A new rule is a new box. The service doesn't need to change.
- Same notation as the data model (Week 3), plus one element (the interface). Read it the same way.



Long parameter lists and boolean flags

```
book(user, room, start, end, true, false, null);
```

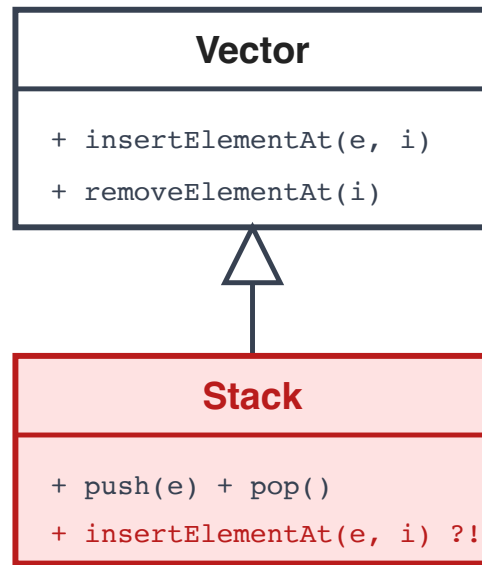
- What does `true` mean here? What about `false`? What is `null` politely declining?
- A growing parameter list usually means a missing type or options object.
- A boolean flag sometimes means two behaviors sharing one function. Giving each its own name might be a reasonable alternative here.

Dead code and speculative generality

- **Dead code:** The method nobody calls, kept just in case. Version control remembers it. Delete it.
 - `BookingPolicy` was one of these (Lab 3).
- **Speculative generality:** The plugin system with one plugin. The abstraction for a second implementation that never came.
- Both make every reader spend time working out that the code does not matter.

Refused bequest

- A subclass inherits an interface and then ignores half of it, or overrides methods to throw.
- Java's own `Stack` extends `Vector`. A stack that lets you `insertElementAt` into the middle. Oops.
- The smell says inheritance was the wrong tool here.
 - The alternative is **composition**. A `Stack` that holds a list as a field can offer `push` and `pop` and nothing else.



Stack inherits every Vector method

(In the JDK since 1.0)

Comments as deodorant

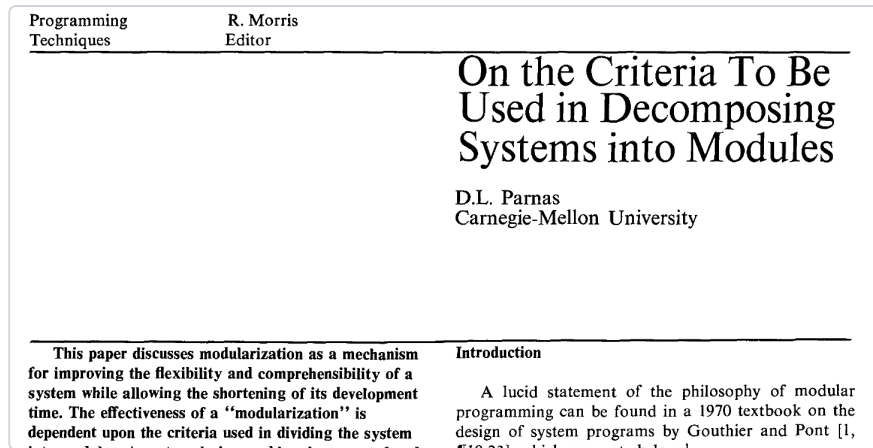
```
// careful: this also updates the waitlist and sends the email  
void process(Booking b) { ... }
```

- When a comment has to explain **what** the code does, the code probably needs a better name or an extraction.
- When the code changes and the comment doesn't, the comment is wrong.
- A good comment explains why the code is the way it is (constraints, surprises, roads not taken).
 - It works like a decision record for a single method.
 - Agents are pretty good at this. (If anything, they overdocument.)

Smells point at principles

What modularity asks for

- A module hides a decision behind an interface. Parnas called it information hiding.
- Done well, we can change what is **inside** without anyone outside noticing.
- Many of today's smells are ways that hiding broke down.



Parnas, "On the Criteria To Be Used in Decomposing Systems into Modules," CACM 15(12), 1972.

Many smells break a principle you already know

Smell	What it violates
God class	Cohesion (one class, one job)
Feature envy	Information Expert (behavior near data)
Inappropriate intimacy	Low coupling (talk through interfaces)
Excessive coupling	Low coupling / Law of Demeter
Cyclic dependencies	Depend in the direction of stability
Hidden dependencies	Hidden coupling / controllability

How to spot them

You can come at a smell from either side.

- From the principle. Pick one (cohesion, Information Expert, low coupling) and look for violations.
- From the code. Find the part that reads oddly and ask which principle it breaks.
- Three questions catch most of it:
 - What is this class **for**? (Can't answer in a sentence? God class.)
 - Does this method mostly use its own fields? (No? Feature envy.)
 - What does this secretly depend on? (Clock, globals, network? Hidden dependencies.)
- Or...

Use a tool

- Linters catch size and complexity (long methods, long parameter lists, God classes). Dependency checkers catch cycles.
- Feature envy, speculative generality, and convention drift are not usually caught by linters.
- Add the linter to the checks your agent runs, so it sees the failure and fixes it before your review.
- You can also ask your agent for a smell pass by name. Treat its findings as candidates to check.

GodClass

Since: PMD 5.0

Priority: Medium (3)

The God Class rule detects the God Class design flaw using metrics. God classes do too many things, are very big and overly complex. They should be split apart to be more object-oriented. The rule uses the detection strategy described in "Object-Oriented Metrics in Practice". The violations are reported against the entire class.

The rule uses metrics to implement its detection strategy. The violation message gives information about the values of these metrics:

- WMC: a class complexity measure, see [WEIGHED_METHOD_COUNT](#)
- ATFD: a measure of how much data external data the class uses, see [ACCESS_TO_FOREIGN_DATA](#)
- TCC: a measure of how tightly related the methods are, see [TIGHT_CLASS_COHESION](#)

The rule identifies a god class by looking for classes which have all of the following properties:

- High WMC
- High ATFD
- Low TCC

See also the reference:

Michale Lanza and Radu Marinescu, *Object-Oriented Metrics in Practice: Using Software Metrics to*

pmd.github.io, Java design rules

New causes for the same smells

Generated code goes wrong in ways we can learn to expect.

**The habits change with each model
that comes out, so let's focus on
the causes.**

Where generated smells come from

- **Missing context:** No author reads the whole repo before every change. What is not in front of the author gets rebuilt or renamed.
- **Underspecified request:** "Add a booking" does not say how general to be, or which inputs can occur. The author fills the gap from habit/intuition.
- **Free volume:** Generating extra code costs the author nothing. Each extra line still has to be tested, kept working, and fit into the next agent's context. (And more work for humans to review, in general.)
- None of the three is special to agents. Human teams produce the same smells the same way, just more slowly.

Duplication over reuse

- The agent re-implements an overlap check that `Room` already has.
- That check was not in the agent's context, so the agent rebuilt it instead of reusing it (missing context).
- Agents with search tools do this less than they used to, but it still happens in large repos.

domain/Room.java (March)

```
boolean isFree(Slot s) {  
    return bookings.stream()  
        .noneMatch(b -> b.overlaps(s));  
}
```

helpers/BookingHelper.java (today's diff)

```
boolean checkAvailable(Room r, Slot s) {  
    for (Booking b : r.getBookings())  
        if (b.start().isBefore(s.end())  
            && s.start().isBefore(b.end())) ...  
}
```

From now on, both have to change together.

Speculative over-abstraction

- We asked for a booking. We got a `BookingStrategyFactoryProvider` with three plugin points.
 - This is speculative generality.
 - It is also an instance of pattern soup.
- We never said how general to be, so the agent guessed (an underspecified request).
- Recall from last time: decoupling a change that never comes is complexity we paid for anyway.

Average day in the life of a Java dev:

Let's add an order interface



Let's add an order factory



Let's make an order factory interface along with an order factory



This is clean code that is easy to maintain and test



programmerhumor.io

Convention drift

- Three services, written in three sessions. Each names its lookup differently and handles "not found" differently.
- Each choice is reasonable on its own. No single file shows what the convention is.
- Each session starts without the conventions the last one used (missing context).
- An instruction file puts the important conventions where each session will read them.

```
Optional<Room> find(RoomId id) // RoomService, Mar: empty if missing
Booking get(BookingId id) // BookingService, Jun: null if missing
Waitlist lookup(RoomId id) // WaitlistService, Sep: throws if missing
```

Phantom complexity

- Code that **looks** sophisticated and does almost nothing.
- It handles cases that can't occur and wraps things that needed no wrapping.
- The agent did not check which inputs can occur (missing context).
- Writing one more guard costs the agent nothing, so it has no reason to leave one out (free volume).

```
Optional<Room> resolveRoom(RoomId id) {  
    if (id == null) return Optional.empty();           // id is @NonNull  
    if (id.value() < 0) return Optional.empty();       // ids can't be negative  
    try {  
        return Optional.ofNullable(rooms.get(id));    // a map lookup  
    } catch (Exception e) { return Optional.empty(); } // get() doesn't throw  
}
```

Three guards and a catch around a one-line map lookup, against inputs that cannot occur.

Comment inflation

```
/** Gets the room id. @returns the room id */  
getRoomId(): string { return this.roomId; }  
  
/** Sets the room id. @param roomId the room id */  
setRoomId(roomId: string): void { this.roomId = roomId; }  
  
// Loop over the bookings  
for (const b of bookings) {
```

- The comments repeat the code in English and explain nothing.
- Many current agents produce this by default, since comments cost them nothing (free volume).
- Keep the comments that explain why. Delete the rest, and/or tell the agent not to write them.

How much of this will still be true next year?

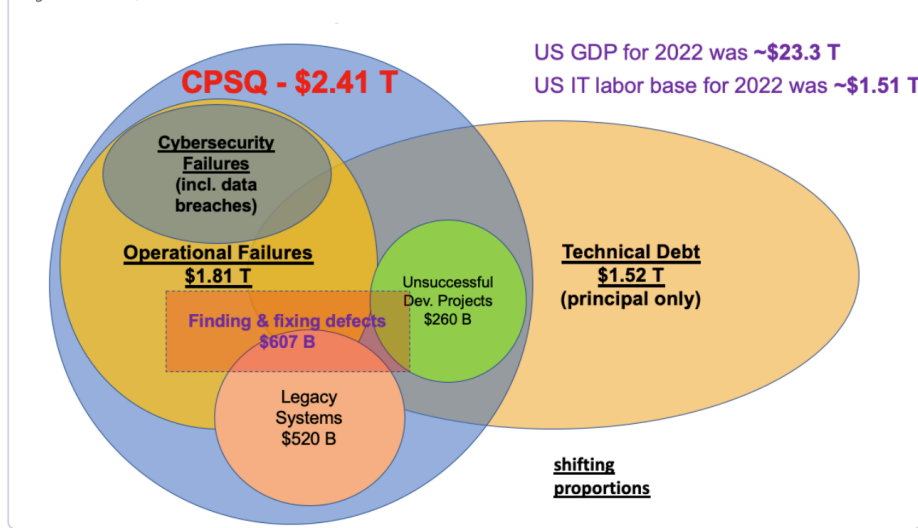
Who knows?

- The five smells are what people commonly report as of, like, now. Some are already fading, and new ones will appear.
- A given model can also err the other way, with too few guards or too few comments.
- The three causes *should* hold much longer, since none of them depends on how capable the model is.
 - But this is a total speculation.
- (For this week's lab, "agent-specific" means one of these five.)

Smells compound into debt

- One smell is cheap. A hundred, each generated in seconds, is **technical debt**.
- Think of it as a loan that charges interest. Every future change has to pay a little extra.
- Agents can take on debt faster than a human team. Code review is where we (painfully) notice the debt we are taking on.
- One estimate of the principal in US software: \$1.52 trillion (CISQ, 2022).

Figure 1-0 CPSQ in US in 2022



CISQ, **The Cost of Poor Software Quality in the US: A 2022 Report**, Fig. 1-0.

When do smells appear?

- A study of 200 open-source projects and half a million commits.
- Most smell instances were there when the file was created. They did not creep in later.
- About 80 percent of the smells survived. Of the ones removed, few went through a refactoring.
- Generated code arrives in bulk, so the best time to catch its smells is when it is created.

When and Why Your Code Starts to Smell Bad (and Whether the Smells Go Away)

Michele Tufano¹, Fabio Palomba^{2,3}, Gabriele Bavota⁴
Rocco Oliveto⁵, Massimiliano Di Penta⁶, Andrea De Lucia³, Denys Poshyvanyk¹

¹The College of William and Mary, Williamsburg, USA, ²Delft University of Technology, The Netherlands

³University of Salerno, Fisciano (SA), Italy, ⁴Università della Svizzera italiana (USI), Switzerland,

⁵University of Molise, Pesche (IS), Italy, ⁶University of Sannio, Benevento (BN), Italy

mtufano@email.wm.edu, fpalomba@unisa.it, gabriele.bavota@usi.ch
rocco.oliveto@unimol.it, dipenta@unisannio.it, adelucia@unisa.it, denys@cs.wm.edu

Abstract—Technical debt is a metaphor introduced by Cunningham to indicate “not quite right code which we postpone making it right”. One noticeable symptom of technical debt is represented by code smells, defined as symptoms of poor design and implementation choices. Previous studies showed the negative impact of code smells on the comprehensibility and maintainability of code. While the repercussions of smells on code quality have been empirically assessed, there is still only anecdotal evidence on *when* and *why* bad smells are introduced, what is their *survivability*, and *how* they are removed by developers. To empirically corroborate such anecdotal evidence, we conducted a large empirical study over the change history of 200 open source projects. This study required the development of a strategy to identify smell-introducing commits, the mining of over half a million of commits, and the manual analysis and classification of over 10K of them. Our findings mostly contradict common wisdom, showing that most of the smell instances are introduced when an artifact is created and not as a result of its evolution. At the same time, 80% of smells survive in the system. Also, among the 20% of removed instances, only 9% is removed as a direct consequence of refactoring operations.

Tufano et al., "When and Why Your Code Starts to Smell Bad (and Whether the Smells Go Away)," IEEE TSE 43(11), 2017 (first version at ICSE 2015).

Which smells do developers notice?

- Developers were shown code with and without smells and asked whether it had a design problem.
- Noticed, and rated severe: God class and long method, among others. Experience made the perception sharper.
- Shrugged at: long parameter list and inappropriate intimacy, among others.
- Some entries are problems most of the time, some rarely.

Do they Really Smell Bad? A Study on Developers' Perception of Bad Code Smells

Fabio Palomba¹, Gabriele Bavota², Massimiliano Di Penta², Rocco Oliveto³, Andrea De Lucia¹
¹University of Salerno, Italy ²University of Sannio, Italy ³University of Molise, Italy

Abstract—In the last decade several catalogues have been defined to characterize bad code smells, i.e., symptoms of poor design and implementation choices. On top of such catalogues, researchers have defined methods and tools to automatically detect and/or remove bad smells. Nevertheless, there is an ongoing debate regarding the extent to which developers perceive bad smells as serious design problems. Indeed, there seems to be a gap between theory and practice, i.e., what is believed to be a problem (theory) and what is actually a problem (practice). This paper presents a study aimed at providing empirical evidence on how developers perceive bad smells. In this study, we showed to developers code entities—belonging to three systems—affected and not by bad smells, and we asked them to indicate whether the code contains a potential design problem, and if any, the nature and severity of the problem. The study involved both original developers from the three projects and outsiders, namely industrial developers and Master students. The results provide insights on characteristics of bad smells not yet explored sufficiently. Also, our findings could guide future research on approaches for the detection and removal of bad smells.

Index Terms—Code Smells, Empirical Study

words, there seem to be a gap between the theory and the practice. For example, a recent study found that some source code files of the Linux Kernel intrinsically have high cyclomatic complexity. However, this is not considered a design or implementation problem by developers [10]. Also, empirical studies indicated that (i) God Classes sporadically changing are not felt as a problem by developers [11]; and (ii) some developers, in particular junior programmers, work better on a version of a system having some classes that centralized the control, i.e., God classes [12]. These results suggest that the presence of bad smells in source code is sometimes tolerable, and part of developers' design choices.

Recently, Yamashita and Moonen [13] performed an exploratory survey aimed at investigating developers knowledge about code smells, by asking questions like "How familiar are you with bad code smells?". Results showed that a large proportion of respondents did not know about bad code

Palomba et al., "Do They Really Smell Bad?", ICSME 2014.

Using the vocabulary

An agent wrote `BookingService`, and we asked for a smell pass on it. Coming soon: rooms in the new building need a cleaning gap between bookings.

```
1  class BookingService {
2      private final RoomRepository rooms;
3      private final Clock clock;
4      private final Mailer mailer;
5      Booking book(User user, RoomId roomId, Slot slot) {
6          Room room = rooms.find(roomId).orElseThrow();
7          if (slot.start().isBefore(clock.now()))
8              throw new PastSlotException();
9          for (Booking b : room.getBookings())
10             if (b.overlaps(slot))
11                 throw new SlotTakenException();
12          Booking booking = room.addBooking(user, slot);
13          mailer.send(user.email(),
14                     "Booked " + room.name() + " at " + slot.start());
15          return booking;
16      }
17  }
```

Smell pass on `BookingService.java`

1. Feature envy (lines 9–11)
book() loops over room.getBookings() to decide availability.
2. Long method (lines 5–16)
Validation, availability, booking, and email in one method.
3. Excessive coupling (lines 2–4)
Depends on RoomRepository, Clock, and Mailer.
4. Duplication (lines 13–14)
The same message text is built in ReminderJob.java:31.

- Is there a finding you would fix first?
- Is there a false positive?
- Is there one you would leave alone?

Which findings matter?

1. Fix first (feature envy). The cleaning gap is an availability rule, and the rule lives in a loop outside `Room`.
 - Anything else that adds a booking would skip it.
2. Leave it (long method). Four steps in a few lines is a method coordinating a use case.
3. A false positive (excessive coupling). Three injected dependencies are explicit dependencies. That is the fix for hidden dependencies.
4. Leave it for now (duplication). It is not in the path of this change, so ask the agent to write it down and move on.

Would your answers change if the next change were a second way to notify people?

A review checklist

Where you review a change closely, you can ask:

- God class? Feature envy? Excessive coupling? Hidden dependencies?
- Did it duplicate something that already exists?
- Did it over-abstract a simple job?
- Does it match the conventions of the code around it?

A green test suite usually does not answer these.

Where to look in a generated change

- Glance at the file list (how many new files, their names, what else changed).
- Let the agent do the searching, and ask it to report what it looked for.
 - You can put the request in your instruction file.
- Spot-check the report. Its claims are candidates to check.
- Read closely only where you expect changes soon, or where a mistake would reach far.
 - For example, an API that clients depend on.

We asked for a confirmation email per booking.

```
$ git diff --stat main
 booking/bookingService.ts      | 14 +-
 notify/NotificationStrategy.ts  | 22 +
 notify/EmailNotificationStrategy.ts | 61 +
 notify/NotificationStrategyFactory.ts | 48 +
 rooms/room.ts                  | 9 +-
 util/dateHelper.ts             | 37 +
```

One email took three new files, there is a new date helper, and `room.ts` changed for an email feature.

After every change, report:

- each file you touched, and why
- new helpers, and what you searched first
- new public signatures
- new hidden inputs (time, env, globals)

Where to look in a generated design

- You are not expected to evaluate every entity, alternative, and tradeoff the agent wrote down.
- Pick the few decisions that matter most.
 - Is it hard to undo? (The data model usually is.)
 - Does something major depend on it?
- Know those few well enough to explain them without your agent.
 - The defense asks about decisions like these.
- For the rest, a general sense that "it is reasonable" is enough.
 - A decision your harness would catch if it were wrong can get a skim, for example.
- You control how much there is to read.
 - You can ask for a shorter summary, or for the choices that would be hardest to reverse.

"Is *this* smell a problem?"

It depends.

On what?

- A smell is just a signal. It tells us where to look.
- Sometimes the pragmatic choice smells a little, and that is fine.
 - For example, a long method in a script we run once.
- The judgment is which smells sit in the path of the changes you expect, or in the way of a check you need.

Don't fix every smell

- Chasing every smell is its own waste, and its own kind of over-engineering.
- Fix the ones in the path of likely change.
- Fix the ones that block a test we need (a hidden dependency, for example).
- When smells intersect, fix the interface first. The rest tends to follow.
- You can also ask your agent to write down the ones you are not fixing. That inventory is a possible start of a refactoring plan.

Summary

- A smell is a symptom in the code. An anti-pattern is the recurring design behind it. Both names say the structure will make the next change harder.
- The classics (God class, feature envy, excessive coupling, hidden dependencies) each violate a principle you already know.
- Generated code shows the same smells for new reasons (missing context, underspecified requests, free volume).
 - Common today: duplication over reuse, speculative over-abstraction, convention drift, phantom complexity, comment inflation.
- Naming them turns "looks fine" into a review checklist for generated code.
- A smell is a signal, not a verdict. Fix the ones that matter. (It depends which.)

Midterm 1 is Wednesday

- Coverage: everything through last week. Verification (workflow and technique) and design fundamentals (what design is, recording it, design for change).
 - Today's material is not in scope.
- Assignments 1 and 2 and the labs covered the same ground.
- How to prepare:
 - Start from each lecture's learning goals, summary, and "What did we talk about last time?" slides. They highlight the key concepts.
 - Work the sample exam questions on Canvas, and redo the in-class exercises.

What's next

W1 A1 out	W2	W3 A1 due	W4	W5 A2 due · M1	W6	W7 A3 due	W8	W9 A4 due	W10 M2	W11	W12 A5 due	W13	W14 A6 due
--------------	----	--------------	----	-------------------	----	--------------	----	--------------	-----------	-----	---------------	-----	---------------

- **Tonight, 11:59pm:** Assignment 2 due.
- **This week:** Assignment 3 comes out.
- **Wednesday:** Midterm 1.
- **Friday, Lab 5:** Anti-pattern critique of generated code. You apply today's checklist to a working, agent-generated module.
- **Next Monday:** Refactoring. Fixing the smells without breaking the behavior.